S^3-Vote: A Reputation-Weighted, Blockchain-Anchored Decentralised Voting System with Adaptive Reconstruction Threshold

A Threshold Cryptography Framework with Publicly Verifiable Secret Sharing and Dynamic Trust Management

Milad Seifi

May 9, 2026

Abstract

Secure electronic voting requires confidentiality of individual ballots, verifiability of the final tally, and resilience against colluding or misbehaving participants — all without relying on a single trusted authority. We present **S^3-Vote** (Secure, Scalable, Sovereign Voting), a decentralised voting system that combines ElGamal threshold encryption, Shamir Secret Sharing, and Publicly Verifiable Secret Sharing (PVSS) with a novel *reputation-weighted adaptive reconstruction threshold*. Each decryption authority accumulates a persistent reputation score derived from their verifiable behaviour across elections. Reputation is converted to a weight, and the effective quorum required for reconstruction adapts automatically: trusted groups need fewer participants; distrusted or heterogeneous groups require more. A two-gate reconstruction policy — enforcing both a cryptographic share count and a reputation weight threshold simultaneously — closes attack vectors that single-threshold schemes leave open. All events, encrypted votes, PVSS commitments, share submissions, cheat evidence, and reputation updates are anchored on a hash-chained blockchain, providing a tamper-evident, non-repudiable audit trail. Our scheme improves upon Shamir's foundational work and Nojoumian's social secret sharing by eliminating the trusted dealer, providing publicly verifiable cheat detection, and introducing a self-governing trust mechanism that adapts dynamically across election rounds.

Contents

1 Introduction

Democratic and organisational decision-making increasingly relies on electronic voting systems. Yet most deployed systems route all trust through a single entity — a government authority, a corporate IT department, or a cloud provider. This creates two fundamental vulnerabilities: a *single point of trust* (the operator can manipulate results undetected) and a *single point of failure* (if the operator is compromised, the election is ruined).

Threshold cryptography distributes trust across N decryption authorities: the private key required to decrypt ballots is split into N shares, and any coalition of at least t shares can reconstruct it, while fewer than t shares reveal nothing. This removes the single point of trust. However, classical threshold schemes treat all shares as equal. A malicious authority submitting a corrupted share causes silent reconstruction failure, and the system has no mechanism to identify or penalise the attacker, nor to adapt future requirements based on observed behaviour.

S^3-Vote addresses these limitations through five integrated contributions:

1. **PVSS-based share verification** — each submitted share is publicly verifiable against an on-chain commitment, making cheating immediately detectable and undeniable.

2. **Persistent reputation scoring** — authority behaviour is scored across Participation, Contribution, and Honesty dimensions and accumulated across elections with exponential smoothing.

3. **Reputation weights** — scores are normalised into per-authority weights, so high-reputation authorities carry proportionally more influence over reconstruction.

4. **Adaptive two-gate quorum** — reconstruction requires satisfying both a cryptographic share count gate *and* a reputation weight gate simultaneously; the weight gate adapts dynamically based on collective trust and evenness.

5. **Blockchain audit trail** — all events are permanently recorded on a hash-chained ledger, providing non-repudiable evidence of every action.

2 Background and Related Work

2.1 ElGamal Encryption

ElGamal encryption [**?**] operates over a cyclic group $\mathbb{Z}_p^*$ with generator g and prime p. A key pair (y, x) is generated where $y = g^x \bmod p$ is the public key and x is the private key. To encrypt a message m (vote option, encoded as a group element):

$$C_1 = g^k \bmod p \tag{1}$$

$$C_2 = m \cdot y^k \bmod p \tag{2}$$

where k is a random ephemeral value. Decryption recovers $m = C_2 \cdot C_1^{-x} \bmod p$. The scheme is semantically secure under the Decisional Diffie-Hellman (DDH) assumption.

In S^3-Vote, the private key x is never held by any single party; it is split via Shamir Secret Sharing before the election begins and reconstructed only after all votes are cast.

2.2 Shamir Secret Sharing

Shamir's (t, n)-threshold scheme [?] splits a secret $s \in \mathbb{Z}_p$ by constructing a random polynomial of degree $t - 1$:

$$f(z) = s + a_1 z + a_2 z^2 + \cdots + a_{t-1} z^{t-1} \pmod p \tag{3}$$

Authority i receives share $s_i = f(i)$. Any t shares uniquely determine f (via Lagrange interpolation) and hence reveal s; any $t-1$ shares are information-theoretically independent of s.

The classical scheme treats all shares as interchangeable: any t-subset suffices. S^3-Vote augments this with reputation weights so that the effective contribution of each share depends on the authority's trust level.

2.3 Publicly Verifiable Secret Sharing (PVSS)

In standard secret sharing, a participant who submits a corrupted share causes reconstruction to fail with no publicly verifiable evidence of wrongdoing. PVSS [?] solves this by publishing *commitments* alongside shares such that anyone can verify correctness without learning the secret.

In S^3-Vote, at election creation, each share s_i generates a commitment:

$$C_i = g^{H(s_i)} \bmod p \tag{4}$$

where H is a cryptographic hash function. These commitments are written to the blockchain before reconstruction begins. When authority i submits $\hat{s}_i$ during reconstruction, the verifier checks $g^{H(\hat{s}_i)} \equiv C_i \pmod p$. A mismatch is on-chain proof of submission of an invalid share.

2.4 Reputation in Secret Sharing: Nojoumian's Scheme

Nojoumian *et al.* [?, ?] introduced the concept of *social secret sharing*, where participant reputation influences the threshold. Their key contributions were recognising that not all participants should be treated equally and that past behaviour should inform future requirements.

However, their scheme has several limitations that S^3-Vote addresses:

- Requires a *trusted dealer* to assign and update reputation scores
- Reputation updates are manual or social (participants evaluate peers), introducing subjectivity and gamesmanship
- No mechanism for public proof of cheating
- Single reconstruction gate (no two-gate policy)
- No protection against reputation concentration
- No blockchain for tamper-evident history
- Designed as a general secret sharing primitive, not a complete voting system

3 System Architecture

3.1 Participants

- **Election Administrator** — creates the election, defines authority set, options, and voter count.
- **Voters** — cast encrypted ballots; their identities are independent of the decryption process.
- **Decryption Authorities** $(A_1, \ldots, A_N)$ — each holds one Shamir share of the private key; collectively responsible for reconstruction after voting closes.
- **Blockchain** — append-only tamper-evident ledger; records all events and acts as the public bulletin board.

3.2 Election Lifecycle

The full protocol proceeds through five phases:

1. **Setup** — key generation, secret splitting, PVSS commitment publication, threshold computation.
2. **Voting** — voters submit ElGamal-encrypted ballots; ciphertexts recorded on-chain.
3. **Reconstruction** — each authority submits their share; PVSS verification runs immediately; coalition weight tracked in real time.
4. **Tally** — once both reconstruction gates are satisfied, the private key is assembled via Lagrange interpolation and all ciphertexts are decrypted.
5. **Reputation Update** — behaviour scores computed for every authority; reputation registry updated; next election's quorum pre-computed from new scores.

If reconstruction becomes *impossible* (too many cheaters and skippers to ever satisfy either gate), the system transitions to a **failed** state: automatic penalties are applied, the blockchain records the failure cause and responsible parties, and the next election is unlocked with an adjusted (higher) quorum.

4 Reputation Model

4.1 Behavioural Scores

After each election, three behavioural dimensions are scored for every authority A_i:

Participation (P_i) Did the authority engage with the protocol?

$$P_i = \begin{cases} 1.0 & \text{submitted valid share} \\ 0.5 & \text{submitted invalid share (cheated)} \\ 0.0 & \text{absent / skipped} \end{cases} \tag{5}$$

Contribution (C_i) How much did their weight cover the per-share average needed?

$$C_i = \begin{cases} \min\left(1, \dfrac{w_i}{\bar{w}_{\text{needed}}}\right) & \text{if submitted valid share} \\ 0.0 & \text{otherwise} \end{cases} \tag{6}$$

where $\bar{w}_{\text{needed}} = \tau/t$ and τ is the quorum, t the Shamir threshold.

Honesty (H_i) Did the authority submit the correct share?

$$H_i = \begin{cases} 1.0 & \text{PVSS verification passed} \\ 0.0 & \text{PVSS verification failed (cheated)} \\ 0.5 & \text{absent — neutral, did not lie} \end{cases} \tag{7}$$

4.2 Reputation Update Formula

The reputation R_i is updated after each election using an exponentially weighted formula:

$$\boxed{R_i^{(\ell+1)} = \delta\left[\lambda R_i^{(\ell)} + (1-\lambda)\left(\alpha P_i + \beta C_i + \gamma H_i\right)\right]} \tag{8}$$

with parameters:

Parameter	Value	Role
α	0.2	Weight of Participation in new score
β	0.3	Weight of Contribution in new score
γ	0.5	Weight of Honesty in new score
λ	0.7	Memory factor (past vs. present)
δ	0.95	Decay factor (small penalty for time elapsed)

Honesty carries the highest weight ($\gamma = 0.5$) because submitting a corrupted share is the most damaging possible action. Memory ($\lambda = 0.7$) ensures that a single bad election does not permanently destroy a long-standing reputation, but sustained misbehaviour accumulates.

4.3 Initial Reputation

New authorities begin at:

$$R_{\text{initial}} = \frac{2}{3} \tag{9}$$

This value is chosen so that a group of entirely fresh, equal-reputation authorities produces a neutral starting quorum of exactly $\lceil N/2 \rceil$ (proven in Section **??**).

4.4 Reputation Weights

Scores are normalised into per-authority weights relative to the group:

$$w_i = \frac{R_i}{\sum_{j=1}^{N} R_j} \cdot N \tag{10}$$

A neutral group (all equal scores) gives $w_i = 1.0$ for all i. An authority with twice the average reputation has $w_i = 2.0$. Weights sum to N by construction.

4.5 Persistence Across Elections

Reputations are stored in a persistent registry keyed by authority name (public key in a production deployment). When an authority participates in a new election, their historical score is loaded automatically. New authorities who have never participated start at R_{initial}. This means the system has *memory*: a long track record of honest behaviour genuinely lowers future quorum requirements.

5 Adaptive Threshold

5.1 Trust and Evenness

The *collective trust* of a group is measured as the product of average reputation and a distributional evenness factor:

$$E = \max\left(0,\ 1 - \frac{\sum_i (R_i - \bar{R})^2}{N\bar{R}^2}\right) \tag{11}$$

$$\tau_{\text{trust}} = \min(1,\ \bar{R} \cdot E) \tag{12}$$

The evenness factor E is close to 1 when all reputations are similar and drops toward 0 when they are highly unequal. This prevents a scenario where a single dominant authority with very high reputation (and correspondingly high weight) could reconstruct alone while the threshold drops — a concentration attack impossible to execute because:

Property 5.1 (Concentration resistance)**.** If one authority's weight approaches $N-1$, evenness $E \to 0$, trust $\tau_{\text{trust}} \to 0$, and the quorum rises to its maximum $N-1$. Therefore no single authority can ever accumulate sufficient weight to meet the quorum alone.

5.2 Two-Piece Linear Adaptive Formula

The effective quorum τ is computed via a two-piece linear function anchored at $\lceil N/2 \rceil$ for neutral trust:

$$\tau_{\text{neutral}} = \left\lceil \frac{N}{2} \right\rceil \tag{13}$$

$$\tau_{\max} = N - 1 \tag{14}$$

$$\tau_{\min} = \max\left(2,\ \left\lceil \frac{N}{3} \right\rceil\right) \tag{15}$$

$$\boxed{\tau = \begin{cases} \tau_{\max} - (\tau_{\max} - \tau_{\text{neutral}}) \cdot \tau_{\text{trust}} \cdot \frac{3}{2} & \text{if } \tau_{\text{trust}} \le \frac{2}{3} \\ \tau_{\text{neutral}} - (\tau_{\text{neutral}} - \tau_{\min}) \cdot (\tau_{\text{trust}} - \frac{2}{3}) \cdot 3 & \text{if } \tau_{\text{trust}} > \frac{2}{3} \end{cases}} \tag{16}$$

Theorem 5.1 (Neutral anchor)**.** For N authorities all at $R_i = \frac{2}{3}$ (the initial reputation), Equation (**??**) yields $\tau = \lceil N/2 \rceil$ exactly.

Proof. All equal reputations give $E = 1$ (zero variance), so $\tau_{\text{trust}} = \bar{R} \cdot 1 = \frac{2}{3}$. Substituting into the first branch: $\tau = \tau_{\max} - (\tau_{\max} - \tau_{\text{neutral}}) \cdot \frac{2}{3} \cdot \frac{3}{2} = \tau_{\max} - (\tau_{\max} - \tau_{\text{neutral}}) = \tau_{\text{neutral}} = \lceil N/2 \rceil$. □

The quorum range for representative group sizes:

N	τ_{neutral}	$\tau_{\min}$ (full trust)	$\tau_{\max}$ (no trust)
3	2	2	2
4	2	2	3
5	3	2	4
6	3	2	5
7	4	3	6
8	4	3	7

6 Two-Gate Reconstruction Policy

Reconstruction is triggered only when *both* conditions are satisfied simultaneously:

$$\underbrace{|\mathcal{S}| \geq t}_{\text{Gate 1: cryptographic}} \qquad \text{AND} \qquad \underbrace{\sum_{i \in \mathcal{S}} w_i \geq \tau}_{\text{Gate 2: trust}} \tag{17}$$

where $\mathcal{S}$ is the set of authorities who submitted valid shares, t is the Shamir threshold (derived as $\lfloor \tau \rfloor$, minimum 2), and τ is the current adaptive quorum.

6.1 Why Two Gates Are Necessary

Gate 1 alone (share count only) An adversary controlling many low-reputation authorities could submit enough shares to satisfy t while their combined weight is far below τ. The weight gate blocks this *flooding attack*.

Gate 2 alone (weight only) A single authority with very high reputation might accumulate weight $\geq \tau$ if the threshold dropped sufficiently. The share count gate (minimum 2) blocks this *monopoly attack*. The evenness mechanism additionally raises τ when one authority dominates, providing a second line of defence.

6.2 Impossibility Detection

The system continuously monitors whether reconstruction is still achievable. Let $\mathcal{V}$ be the set of valid-share submitters, $\mathcal{F}$ the set of authorities who have acted (valid, invalid, or skipped), and $\mathcal{U} = \{A_i\} \setminus \mathcal{F}$ the remaining free authorities.

$$\text{Impossible} \iff |\mathcal{V}| + |\mathcal{U}| < t \ \text{ OR } \ \sum_{i \in \mathcal{V}} w_i + \sum_{i \in \mathcal{U}} w_i < \tau \tag{18}$$

If impossible, the system transitions to a `failed` state immediately — before waiting for remaining authorities to act — applies automatic penalties to all responsible parties, and records the event on the blockchain.

7 Blockchain Integration

7.1 Role of the Blockchain

The blockchain in S^3-Vote serves as a **tamper-evident public bulletin board**. It does not provide consensus (single-node prototype) but provides the structural guarantees needed for the protocol:

1. **Commitment anchoring** — PVSS commitments are published before any share submission, preventing retroactive commitment substitution.
2. **Vote integrity** — encrypted ballots are written before reconstruction begins; the tally cannot be manipulated retroactively.
3. **Non-repudiation** — share submissions and failures are permanently recorded with the submitting authority's identity; a cheater cannot deny the act.
4. **Reputation audit trail** — every reputation update is recorded with its round number; the full history is independently verifiable.

7.2 Block Structure

Each block B_k contains:

- Index k, timestamp
- Data payload (event dictionary)
- $\text{hash}_k = \text{SHA-256}(\text{index} \,\|\, \text{timestamp} \,\|\, \text{data} \,\|\, \text{hash}_{k-1})$

Altering any block invalidates all subsequent hashes, making tampering detectable by any observer who holds a copy of the chain.

7.3 Events Recorded

Event	Payload
`election_start`	title, options, authorities, quorum, PVSS commitments
`vote_cast`	round, ElGamal ciphertext (C_1, C_2)
`voting_closed`	round, total votes cast
`share_submitted`	round, authority, coalition weight at submission
`share_invalid`	round, authority (PVSS verification failed)
`share_skipped`	round, authority
`reconstruction_complete`	round, coalition, coalition weight, tally
`election_failed`	round, valid shares, cheaters, skippers
`reputation_update`	round, new reputation scores, new quorum

7.4 Production Deployment Note

The prototype implements the blockchain as an in-memory hash-chained list on a single node. In a production deployment, this would be replaced by a distributed ledger (e.g., Ethereum or Hyperledger Fabric), where no single party controls the chain and tamper-evidence is enforced by network consensus. The data model and cryptographic properties are identical; only the storage and consensus layer changes.

8 Security Analysis

8.1 Vote Confidentiality

Individual votes remain confidential until reconstruction because: (a) the ElGamal private key is split across N authorities; (b) any $t-1$ or fewer shares reveal no information about the key (information-theoretic security of Shamir's scheme); (c) ciphertexts on the blockchain are computationally indistinguishable from random under the DDH assumption.

8.2 Tally Integrity

Once voting closes, the `voting_closed` block is chained. Any attempt to add, remove, or modify a `vote_cast` block breaks the hash chain from that point forward, detectable by any holder of the chain.

8.3 Cheat Detection and Accountability

PVSS commitments are written at election creation. A cheating authority A_i who submits $\hat{s}_i \neq s_i$ will fail the commitment check $g^{H(\hat{s}_i)} \not\equiv C_i \pmod{p}$. The failure, the authority identity, and the round are recorded as a `share_invalid` block — public, permanent, and hash-chained into the ledger before A_i can dispute it.

8.4 Sybil Resistance

The reputation system is resistant to Sybil attacks (creating many fake identities) because new authorities start at $R_{\text{initial}} = 2/3$ and must build a track record over multiple elections to gain elevated weight. A flood of new identities in a round does not lower the quorum — it raises it, because many new authorities with neutral reputation increase N and thus $\tau_{\text{neutral}} = \lceil N/2 \rceil$.

8.5 Collusion Resistance

A colluding coalition of k low-reputation authorities cannot satisfy the weight gate unless $\sum_{i \in \mathcal{C}} w_i \geq \tau$. Since low-reputation authorities have $w_i < 1$, they need more members than a trusted coalition — and the evenness factor raises τ further when the honest authorities' reputations are heterogeneous.

8.6 Concentration Attack

As proven in Property 1, a single authority cannot accumulate enough weight to reconstruct alone regardless of their reputation, because the evenness factor E approaches zero as weight concentrates, driving the quorum to $\tau_{\max} = N - 1$.

9 Comparison with Prior Work

9.1 Against Shamir's Threshold Scheme (1979)

Shamir's original (t, n)-threshold scheme [**?**] treats every share as equal and provides no mechanism for identifying or penalising misbehaving participants. Key differences:

Property	Shamir (1979)	S^3-Vote
Share equality	All equal	Reputation-weighted
Cheat detection	None	PVSS — immediate, on-chain proof
Threshold	Fixed	Adaptive (reputation-driven)
Failed round handling	Not addressed	Automatic penalty + recovery
Authority accountability	None	Persistent, cross-round reputation

9.2 Against Nojoumian's Social Secret Sharing

Nojoumian *et al.* [**?**, **?**] introduced reputation into secret sharing but left several critical gaps:

Property	Nojoumian	S^3-Vote
Trusted dealer	Required	None
Reputation assignment	Manual / social (subjective)	Formula-based, automatic
Cheat detection	Implicit (failure only)	PVSS — public, on-chain
Reconstruction gate	Single (weighted shares)	Dual (shares AND weight)
Concentration protection	None	Evenness factor
Neutral starting quorum	Ad hoc	$\lceil N/2 \rceil$ (proven)
Failed election recovery	Not addressed	Automatic penalties + recovery
Audit trail	Database	Blockchain (hash-chained)
Application domain	General secret sharing	Complete voting system

Trusted Dealer Elimination

Nojoumian's scheme requires a trusted dealer to initialise reputation scores and arbitrate updates. This reintroduces the central trust problem that threshold cryptography exists to solve. S^3-Vote derives reputation entirely from verifiable on-chain behaviour — no coordinator, no subjectivity.

Objective vs. Social Reputation

In Nojoumian's model, authorities evaluate each other — a social process that is gameable (coalitions can inflate each other's scores). In S^3-Vote, scores are computed deterministically from Equation (**??**) applied to PVSS-verified outcomes. There is no way to inflate a reputation except by genuinely behaving well.

Two-Gate Policy

Nojoumian uses a single adjusted threshold. S^3-Vote enforces Equation (**??**): both the cryptographic share count and the trust weight must be satisfied. This closes the flooding attack (many low-weight shares) and the monopoly attack (one very high-weight authority) that single-gate schemes cannot prevent.

Provable Neutral Anchor

Nojoumian does not derive a principled starting threshold. S^3-Vote proves (Theorem 1) that fresh authorities always begin at the majority quorum $\lceil N/2 \rceil$, providing a principled, consistent baseline regardless of group size.

10 Prototype Implementation

The S^3-Vote prototype is implemented in Python (backend) and JavaScript (single-page frontend), comprising the following modules:

`crypto/encryption.py` ElGamal key generation, vote encryption and decryption.

`crypto/shamir.py` Polynomial-based (t, n)-threshold secret splitting and Lagrange interpolation reconstruction.

`crypto/pvss.py` Commitment generation and share verification.

`reputation/reputation_engine.py` Implementation of Equation (**??**).

`reputation/threshold.py` Implementation of Equations (**??**)–(**??**).

`blockchain/blockchain.py` Hash-chained block structure with SHA-256 linking.

`api/app.py` Flask REST API orchestrating the full election lifecycle.

`api/templates/index.html` Web dashboard for election management, vote simulation, share submission, and reputation visualisation.

The dashboard allows an operator to create elections with arbitrary authority sets, simulate voting, and control each authority's share submission (valid, invalid, or skipped) to observe the quorum and reputation dynamics in real time. Reputation persists across sessions via the in-memory registry and reflects accurately in the quorum computation for subsequent elections.

11 Example Walkthrough

We trace a two-round scenario with $N = 7$ authorities.

Round 1 — Initial State

All 7 authorities start at $R_i = 2/3$. By Theorem 1, the quorum is $\tau = \lceil 7/2 \rceil = 4$. Suppose 5 submit valid shares, 1 cheats, and 1 skips. The coalition of 5 valid submitters achieves weight $5 \times 1.0 = 5.0 \geq 4$ and share count $5 \geq t = 4$. Reconstruction succeeds.

After the round, reputation updates via Equation (**??**):

$$\begin{aligned} R_{\text{honest}} &\approx 0.95[0.7(2/3) + 0.3(1.0)] \approx 0.728 \\ R_{\text{cheater}} &\approx 0.95[0.7(2/3) + 0.3(0.5 \cdot 0.2 + 0 \cdot 0.3 + 0 \cdot 0.5)] \approx 0.472 \\ R_{\text{skipper}} &\approx 0.95[0.7(2/3) + 0.3(0 \cdot 0.2 + 0 \cdot 0.3 + 0.5 \cdot 0.5)] \approx 0.515 \end{aligned}$$

Round 2 — Reputations Loaded from Registry

The new reputations $\{0.728 \times 5,\ 0.472,\ 0.515\}$ are loaded. The honest majority has higher average rep but the two underperformers reduce evenness. Trust $\tau_{\text{trust}} \approx 0.67$ remains near neutral, so the quorum adjusts to approximately $\tau \approx 4.07$ — slightly *above* neutral, reflecting that the group is less cohesive than before. The cheater and skipper must now work against a marginally stricter quorum, while repeated honest behaviour across subsequent rounds will drive it back down toward 3.

12 Limitations and Future Work

Single-node blockchain The prototype runs on one server. Production deployment requires a distributed ledger with consensus.

Simulated authority clients All authority actions are simulated by one administrator in the prototype. A production system gives each authority an independent client that holds their private key and signs share submissions.

Cryptographic identity Authority names are currently plain strings. In production, each name maps to a public key; share submissions are signed by the corresponding private key and verified on-chain, preventing impersonation.

Parameter sensitivity The reputation parameters $(\alpha, \beta, \gamma, \lambda, \delta)$ are set by design rationale. Empirical calibration across realistic election scenarios would strengthen the parameter choices.

Formal security proof A rigorous game-based security proof under standard cryptographic assumptions (DDH, random oracle) is left for future work.

13 Conclusion

S^3-Vote demonstrates that threshold-cryptographic voting can be made self-governing: no trusted dealer, no static parameters, no silent failures. By anchoring PVSS commitments and all authority actions on a blockchain, every act of honesty or betrayal becomes a permanent, public, verifiable record. By feeding that record into a reputation formula and propagating reputations across elections, the system develops institutional memory. And by translating that memory into an adaptive two-gate quorum, it turns good behaviour

into a concrete, quantifiable reward — fewer participants needed to reconstruct — and bad behaviour into a concrete cost — a higher, stricter quorum for everyone.

The core theoretical contribution is the proof that the neutral quorum is always $\lceil N/2 \rceil$ regardless of group size, that evenness prevents weight concentration, and that the dual-gate policy closes attack vectors that prior single-threshold schemes cannot address.

Compared to Shamir's foundational scheme, S^3-Vote adds accountability, verifiability, and adaptivity. Compared to Nojoumian's social secret sharing, it eliminates the trusted dealer, replaces subjective social reputation with objective on-chain evidence, introduces dual-gate reconstruction, and provides a complete, deployable voting system rather than a secret sharing primitive.

References

[1] A. Shamir, "How to Share a Secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[2] T. ElGamal, "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469–472, 1985.

[3] B. Schoenmakers, "A Simple Publicly Verifiable Secret Sharing Scheme and its Application to Electronic Voting," in *Advances in Cryptology — CRYPTO 1999*, Lecture Notes in Computer Science, vol. 1666, pp. 148–164, Springer, 1999.

[4] M. Nojoumian and D. R. Stinson, "Social Secret Sharing in Cloud Computing Using a New Trust Function," in *Proceedings of the 10th Annual Conference on Privacy, Security and Trust (PST)*, pp. 161–167, IEEE, 2012.

[5] M. Nojoumian, "Novel Secret Sharing and Commitment Schemes for Cryptographic Applications," Ph.D. dissertation, University of Waterloo, 2012. (Extended work published through 2018.)

[6] T. P. Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing," in *Advances in Cryptology — CRYPTO 1991*, Lecture Notes in Computer Science, vol. 576, pp. 129–140, Springer, 1991.

[7] Y. Desmedt and Y. Frankel, "Threshold Cryptosystems," in *Advances in Cryptology — CRYPTO 1989*, Lecture Notes in Computer Science, vol. 435, pp. 307–315, Springer, 1990.

[8] R. Cramer, R. Gennaro, and B. Schoenmakers, "A Secure and Optimally Efficient Multi-Authority Election Scheme," in *Advances in Cryptology — EUROCRYPT 1997*, Lecture Notes in Computer Science, vol. 1233, pp. 103–118, Springer, 1997.

[9] D. Chaum, C. Crépeau, and I. Damgård, "Multiparty Unconditionally Secure Protocols," in *Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 11–19, ACM, 1988.